

LAPORAN PRAKTIKUM
PEMROGRAMAN BERORIENTASI OBJEK
MODUL XI
ERROR HANDLING



Disusun Oleh:

Arif Mufti Tharsa (105224041)

PROGRAM STUDI ILMU KOMPUTER
FAKULTAS SAINS DAN KOMPUTER
UNIVERSITAS PERTAMINA

2026

I. Pendahuluan

Pada THT Modul 11 ini, saya membuat program Java sederhana tentang Sistem Reservasi Tiket Kereta Api yang diberi nama JAVA EXPRESS. Program ini berjalan lewat terminal atau CLI, jadi pengguna bisa memilih menu untuk melihat jadwal kereta, memesan tiket, atau keluar dari sistem.

Program ini terdiri dari beberapa class, yaitu KeretaApi, SistemReservasi, Main, dan beberapa class exception buatan sendiri. Class KeretaApi dipakai untuk menyimpan data kereta, SistemReservasi dipakai untuk mengatur proses reservasi, sedangkan Main dipakai untuk menjalankan menu utama. Untuk error khusus, program memakai DataPenumpangTidakValidException, RuteTidakDitemukanException, dan TiketHabisException.

Materi utama yang digunakan adalah Error Handling. Program ini menangani beberapa kondisi error, seperti input menu yang bukan angka, NIK yang tidak valid, kode kereta yang tidak ditemukan, jumlah tiket yang tidak masuk akal, dan jumlah tiket yang lebih besar dari sisa kursi. Selain itu, saya juga menambahkan kode booking sederhana sebagai tambahan agar hasil pemesanan terlihat lebih lengkap.

II. Variabel

No	Nama Variabel	Tipe Data	Fungsi
1	kodeKereta	String	Menyimpan kode kereta, misalnya K01 atau K02.
2	namaKereta	String	Menyimpan nama kereta seperti Argo Bromo atau Parahyangan.
3	rute	String	Menyimpan rute perjalanan kereta, misalnya JKT - SBY.
4	sisaKursi	int	Menyimpan jumlah kursi yang masih tersedia.
5	daftarKereta	ArrayList<KeretaApi>	Menyimpan daftar objek kereta yang tersedia di sistem.
6	nomorBooking	int	Menyimpan nomor urut untuk membuat kode booking sederhana.
7	nik	String	Menyimpan NIK penumpang yang akan divalidasi.
8	namaPenumpang	String	Menyimpan nama penumpang yang memesan tiket.
9	jumlahTiket	int	Menyimpan jumlah tiket yang ingin dipesan.
10	kereta	KeretaApi	Objek sementara untuk menyimpan hasil pencarian kereta.
11	input	Scanner	Digunakan untuk mengambil input dari pengguna.
12	reservasi	SistemReservasi	Objek utama untuk menjalankan fitur reservasi.
13	jalan	boolean	Mengatur apakah menu program masih berjalan atau sudah berhenti.
14	pilihan	int	Menyimpan pilihan menu yang dimasukkan pengguna.

III. Constructor dan Method

No	Nama Metode	Jenis Metode	Fungsi
1	DataPenumpangTidakValidException(String)	Constructor	Mengisi pesan error untuk data penumpang yang tidak valid.
2	RuteTidakDitemukanException(String)	Constructor	Mengisi pesan error jika kode kereta tidak ditemukan.
3	TiketHabisException(String, int)	Constructor	Mengisi pesan error sekaligus menyimpan nama kereta dan sisa kursi.
4	getNamaKereta()	Accessor	Mengambil nama kereta dari exception TiketHabisException.
5	getSisaKursi()	Accessor	Mengambil jumlah sisa kursi dari exception TiketHabisException.
6	KeretaApi(String, String, String, int)	Constructor	Mengisi data awal kereta berupa kode, nama, rute, dan sisa kursi.
7	tampilkanInfo()	Procedural	Menampilkan detail informasi kereta ke konsol.
8	kurangiKursi(int)	Procedural	Mengurangi sisa kursi setelah tiket berhasil dipesan.
9	SistemReservasi()	Constructor	Mengisi data awal kereta yang tersedia di sistem.
10	lihatJadwal()	Procedural	Menampilkan semua jadwal kereta yang ada di daftarKereta.
11	validasiNik(String)	Procedural	Mengecek apakah NIK berjumlah 16 digit dan hanya berisi angka.
12	validasiNama(String)	Procedural	Mengecek apakah nama penumpang kosong atau tidak.
13	cariKereta(String)	Function	Mencari kereta berdasarkan kode kereta yang dimasukkan.

14	pesanTiket(...)	Procedural	Memproses pemesanan tiket, mulai dari validasi sampai pengurangan kursi.
15	main(String[] args)	Main Method	Menjalankan menu utama program JAVA EXPRESS.

IV. Dokumentasi dan Pembahasan Code

DataPenumpangTidakValidException.java

```
public class DataPenumpangTidakValidException extends RuntimeException {  
    public DataPenumpangTidakValidException(String pesan) {  
        super(pesan);  
    }  
}
```

Class DataPenumpangTidakValidException dipakai untuk menangani data penumpang yang tidak sesuai aturan. Karena class ini extends RuntimeException, error ini termasuk unchecked exception. Di program ini, exception ini dipakai saat NIK tidak berjumlah 16 digit, NIK berisi huruf, nama penumpang kosong, atau jumlah tiket kurang dari sama dengan 0.

RuteTidakDitemukanException.java

```
public class RuteTidakDitemukanException extends Exception {  
    public RuteTidakDitemukanException(String pesan) {  
        super(pesan);  
    }  
}
```

Class RuteTidakDitemukanException dipakai saat kode kereta yang dimasukkan pengguna tidak ada di daftar kereta. Karena class ini extends Exception, maka termasuk checked exception dan harus ditangani menggunakan try-catch atau dideklarasikan dengan throws.

TiketHabisException.java

```
public class TiketHabisException extends Exception {  
    String namaKereta;  
    int sisaKursi;  
  
    public TiketHabisException(String namaKereta, int sisaKursi) {  
        super("Tiket tidak mencukupi untuk kereta " + namaKereta);  
        this.namaKereta = namaKereta;  
        this.sisaKursi = sisaKursi;  
    }  
  
    public String getNamaKereta() {  
        return namaKereta;  
    }  
  
    public int getSisaKursi() {  
        return sisaKursi;  
    }  
}
```

Class `TiketHabisException` dipakai saat jumlah tiket yang dipesan lebih banyak dari sisa kursi. Di dalam class ini ada `namaKereta` dan `sisaKursi` supaya pesan error bisa lebih jelas. Method `getNamaKereta()` dan `getSisaKursi()` dipakai di Main untuk menampilkan info tambahan saat tiket tidak cukup.

KeretaApi.java

```
public class KeretaApi {
    String kodeKereta;
    String namaKereta;
    String rute;
    int sisaKursi;

    public KeretaApi(String kodeKereta, String namaKereta, String rute, int
sisaKursi) {
        this.kodeKereta = kodeKereta;
        this.namaKereta = namaKereta;
        this.rute = rute;
        this.sisaKursi = sisaKursi;
    }

    public void tampilkanInfo() {
        System.out.println("Kode Kereta: " + kodeKereta);
        System.out.println("Nama Kereta: " + namaKereta);
        System.out.println("Rute: " + rute);
        System.out.println("Sisa Kursi: " + sisaKursi);
        System.out.println();
    }

    public void kurangiKursi(int jumlah) {
        sisaKursi -= jumlah;
    }
}
```

Class `KeretaApi` berfungsi sebagai tempat menyimpan data kereta. Data yang disimpan adalah `kodeKereta`, `namaKereta`, `rute`, dan `sisaKursi`. Constructor dipakai supaya data kereta langsung terisi saat objek dibuat. Method `tampilkanInfo()` dipakai untuk menampilkan detail kereta, sedangkan `kurangiKursi()` dipakai untuk mengurangi kursi setelah pemesanan berhasil.

SistemReservasi.java

```
import java.util.ArrayList;
```

```

public class SistemReservasi {
    ArrayList<KeretaApi> daftarKereta = new ArrayList<>();
    int nomorBooking = 1;

    public SistemReservasi() {
        daftarKereta.add(new KeretaApi("K01", "Argo Bromo", "JKT - SBY", 50));
        daftarKereta.add(new KeretaApi("K02", "Parahyangan", "JKT - BDG", 15));
    }

    public void lihatJadwal() {
        System.out.println("\n--- Jadwal Kereta Java Express ----");
        for (int i = 0; i < daftarKereta.size(); i++) {
            daftarKereta.get(i).tampilkanInfo();
        }
    }

    public void validasiNik(String nik) {
        if (nik.length() != 16) {
            throw new DataPenumpangTidakValidException("NIK harus berjumlah 16 digit");
        }

        for (int i = 0; i < nik.length(); i++) {
            if (!Character.isDigit(nik.charAt(i))) {
                throw new DataPenumpangTidakValidException("NIK hanya boleh berisi angka");
            }
        }
    }

    public void validasiNama(String namaPenumpang) {
        if (namaPenumpang.trim().isEmpty()) {
            throw new DataPenumpangTidakValidException("Nama penumpang tidak boleh kosong");
        }
    }

    public KeretaApi cariKereta(String kodeKereta) {
        for (int i = 0; i < daftarKereta.size(); i++) {
            KeretaApi kereta = daftarKereta.get(i);
            if (kereta.kodeKereta.equalsIgnoreCase(kodeKereta)) {
                return kereta;
            }
        }
        return null;
    }

    public void pesanTiket(String kodeKereta, String nik, String namaPenumpang,
int jumlahTiket)
        throws RuteTidakDitemukanException, TiketHabisException {

```

```

        validasiNik(nik);
        validasiNama(namaPenumpang);
        if (jumlahTiket <= 0) {
            throw new DataPenumpangTidakValidException("Jumlah tiket harus lebih
dari 0");
        }

        KeretaApi kereta = cariKereta(kodeKereta);
        if (kereta == null) {
            throw new RuteTidakDitemukanException("Kode kereta " + kodeKereta + "
tidak ditemukan");
        }

        if (jumlahTiket > kereta.sisaKursi) {
            throw new TiketHabisException(kereta.namaKereta, kereta.sisaKursi);
        }
        kereta.kurangiKursi(jumlahTiket);
        String kodeBooking = "JEX-" + nomorBooking;
        nomorBooking++;

        System.out.println("\n--- Pemesanan Berhasil ---");
        System.out.println("Kode Booking: " + kodeBooking);
        System.out.println("Nama Penumpang: " + namaPenumpang);
        System.out.println("NIK: " + nik);
        System.out.println("Kereta: " + kereta.namaKereta);
        System.out.println("Kode Kereta: " + kereta.kodeKereta);
        System.out.println("Rute: " + kereta.rute);
        System.out.println("Jumlah Tiket: " + jumlahTiket);
        System.out.println("Sisa Kursi: " + kereta.sisaKursi);
    }
}

```

Class SistemReservasi menjadi pusat dari proses reservasi. Di constructor, sistem langsung mengisi dua data kereta awal, yaitu Argo Bromo dan Parahyangan. Method lihatJadwal() menampilkan semua kereta menggunakan perulangan biasa. Method validasiNik() mengecek panjang NIK dan isi karakternya. Method validasiNama() mengecek agar nama penumpang tidak kosong. Method cariKereta() mencari kereta berdasarkan kode, sedangkan pesanTiket() mengatur proses pemesanan dari awal sampai berhasil atau gagal.

Main.java

```

import java.util.InputMismatchException;
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {

```

```

Scanner input = new Scanner(System.in);
SistemReservasi reservasi = new SistemReservasi();

boolean jalan = true;
try {
    while (jalan) {
        try {
            System.out.println("---- JAVA EXPRESS -----");
            System.out.println("1. Lihat Jadwal Kereta");
            System.out.println("2. Pesan Tiket");
            System.out.println("3. Keluar");
            System.out.print("Pilih menu: ");

            int pilihan = input.nextInt();
            input.nextLine();

            switch (pilihan) {
                case 1:
                    reservasi.lihatJadwal();
                    break;

                case 2:
                    System.out.println("\n---- Form Pemesanan Tiket
---");

                    System.out.print("Masukkan kode kereta: ");
                    String kodeKereta = input.nextLine();
                    System.out.print("Masukkan NIK penumpang: ");
                    String nik = input.nextLine();
                    System.out.print("Masukkan nama penumpang: ");
                    String namaPenumpang = input.nextLine();
                    System.out.print("Masukkan jumlah tiket: ");
                    int jumlahTiket = input.nextInt();
                    input.nextLine();
                    reservasi.pesanTiket(kodeKereta, nik, namaPenumpang,
jumlahTiket);

                    break;

                case 3:
                    System.out.println("Anda memilih keluar dari
sistem");

                    jalan = false;
                    break;
                default:
                    System.out.println("Pilihan menu tidak valid");
            }

        } catch (InputMismatchException e) {
            System.out.println("Error: Input harus berupa angka");
            input.nextLine();
        } catch (DataPenumpangTidakValidException e) {

```

```

        System.out.println("Error data penumpang: " +
e.getMessage());
    } catch (RuteTidakDitemukanException e) {
        System.out.println("Error rute: " + e.getMessage());
    } catch (TiketHabisException e) {
        System.out.println("Error tiket: " + e.getMessage());
        System.out.println("Nama Kereta: " + e.getNamaKereta());
        System.out.println("Sisa Kursi: " + e.getSisaKursi());
    }
}
} finally {
    input.close();
}
}
}

```

Class Main dipakai untuk menjalankan program lewat terminal. Di sini ada menu utama yang terus berjalan selama variabel jalan bernilai true. Pengguna bisa melihat jadwal, memesan tiket, atau keluar dari sistem. Bagian try-catch dipakai agar program tidak langsung berhenti saat terjadi error. InputMismatchException dipakai untuk menangani input yang seharusnya angka, tetapi diisi huruf. Custom exception dari SistemReservasi juga ditangkap di sini agar pesan error bisa ditampilkan dengan lebih jelas.

Pembahasan Error Handling

Pada program ini, error handling dipakai supaya aplikasi tidak langsung crash saat pengguna salah input atau saat proses pemesanan tidak bisa dilanjutkan. Blok try-catch di Main menangani beberapa jenis error, seperti InputMismatchException untuk input angka yang salah, DataPenumpangTidakValidException untuk data penumpang yang tidak sesuai, RuteTidakDitemukanException untuk kode kereta yang tidak ada, dan TiketHabisException untuk kondisi kursi tidak mencukupi.

Keyword throw dipakai di dalam class SistemReservasi untuk melempar error secara manual ketika data tidak sesuai aturan. Sementara itu, keyword throws dipakai pada method pesanTiket() karena method tersebut bisa melempar checked exception, yaitu RuteTidakDitemukanException dan TiketHabisException. Blok finally di Main dipakai untuk menutup Scanner setelah program selesai berjalan.

Program ini juga punya tambahan sederhana berupa kode booking. Kode booking dibuat dari nomorBooking, lalu dinaikkan setiap kali pemesanan berhasil. Jadi hasil pemesanan tidak hanya menampilkan data penumpang dan kereta, tetapi juga kode booking seperti JEX-1, JEX-2, dan seterusnya.

V. Kesimpulan

Program JAVA EXPRESS ini sudah menerapkan konsep Error Handling secara sederhana. Class KeretaApi dipakai sebagai model data kereta, SistemReservasi dipakai untuk mengatur proses reservasi tiket, dan Main dipakai untuk menjalankan menu utama di terminal.

Custom exception digunakan untuk menangani error yang sesuai dengan kasus program. DataPenumpangTidakValidException dipakai untuk NIK, nama, atau jumlah tiket yang tidak valid. RuteTidakDitemukanException dipakai saat kode kereta tidak ditemukan. TiketHabisException dipakai saat jumlah tiket yang dipesan lebih banyak dari sisa kursi.

Secara keseluruhan, program ini sudah sesuai untuk latihan error handling karena bisa menangani input salah, validasi data, checked exception, unchecked exception, throw, throws, try-catch, dan finally. Program juga ditambah kode booking sederhana supaya hasil pemesanan terlihat lebih lengkap.